

start-dfs.sh

start-yarn.sh

jps

cd ~/mapreduce

code . or go to the ubuntu home folder or search for mapreduce folder keep the mapper.py, reducer.py and test_input.txt files there

chmod +x mapper.py reducer.py – gives permission to run files to hdfs

hadoop fs -rm -r /output_result

hadoop fs -mkdir -p /input

hadoop fs -put -f test_input.txt /input/

hadoop fs -ls /input

find /usr/local/hadoop -name "hadoop-streaming*.jar"

COPY THE FIRST FILE PATH

hadoop jar [PASTE_JAR_PATH_HERE] \

-file mapper.py -mapper "python3 mapper.py" \

-file reducer.py -reducer "python3 reducer.py" \

-input /input/test_input.txt \

-output /output_result \

-cmdenv TARGET_WORD=apple

hadoop fs -cat /output_result/part-00000

hadoop fs -get /output_result/part-00000 ./final_count.txt

stop-all.sh

touch mapper.py – makes file

nano mapper.py – opens file

Delete the failed output folder:

hadoop fs -rm -r /output_result

Phase 1: Wake up the Cluster

First, make sure the Hadoop "engines" are running.

Bash

Start storage services

start-dfs.sh

Start resource management services

start-yarn.sh

Verify everything is running (Look for NameNode, DataNode, etc.)

jps

Phase 2: Project Setup

Move into your folder and make sure your scripts are ready to go.

Bash

Go to your project folder

cd ~/mapreduce

Open VS Code to check your python code

code .

IMPORTANT: Give Ubuntu permission to run your scripts

chmod +x mapper.py reducer.py

Phase 3: The HDFS "Cleanup & Upload"

Hadoop will fail if the output folder already exists, so we always clean up first.

Bash

1. Delete old output from previous runs

```
hadoop fs -rm -r /output_result
```

2. Create input folder in Hadoop (if not already there)

```
hadoop fs -mkdir -p /input
```

3. Upload your text file from your computer to Hadoop

```
hadoop fs -put -f test_input.txt /input/
```

4. Verify the file is in Hadoop

```
hadoop fs -ls /input
```

Phase 4: Find the "Bridge" (Streaming JAR)

You need the exact path to the Streaming JAR to run Python.

Bash

```
find /usr/local/hadoop -name "hadoop-streaming*.jar"
```

Copy the first path that ends in .jar.

Phase 5: Execute the MapReduce Job

Paste the JAR path where indicated below. Note: The \ allows you to write one command over multiple lines for better readability.

Bash

```
hadoop jar [PASTE_JAR_PATH_HERE] \  
-file mapper.py -mapper "python3 mapper.py" \  
-file reducer.py -reducer "python3 reducer.py" \  
-input /input/test_input.txt \  

```

```
-output /output_result \
```

```
-cmdenv TARGET_WORD=apple
```

Phase 6: Show the Results

Once the terminal says "Job completed successfully," show your work.

To show the result in the terminal:

Bash

```
hadoop fs -cat /output_result/part-00000
```

To download the file to your local folder to show the examiner:

Bash

```
hadoop fs -get /output_result/part-00000 ./final_count.txt
```

💡 Quick Troubleshooting for the Exam:

- **If jps shows nothing:** Your Hadoop variables aren't loaded. Run `source ~/.bashrc`.
- **If it says "Permission Denied":** You forgot `chmod +x mapper.py reducer.py`.
- **If it says "Output directory already exists":** Run `hadoop fs -rm -r /output_result`.
- **If the count is 0:** Check if the word in `-cmdenv TARGET_WORD=...` actually exists in your `test_input.txt`.

Good luck with your exam

Mapper.py

```
#!/usr/bin/env python3
```

```
import sys
```

```
# The target word we are looking for (passed as an environment variable)
```

```
import os
```

```
target_word = os.environ.get('TARGET_WORD', 'hadoop').lower()
```

```
for line in sys.stdin:
```

```
    # Clean and split the line
```

```
    words = line.strip().split()
```

```
    for word in words:
```

```
        # Remove punctuation and lowercase
```

```
        clean_word = "".join(filter(str.isalnum, word)).lower()
```

```
        if clean_word == target_word:
```

```
            print(f"{clean_word}\t1")
```

```
reducer.py
```

```
#!/usr/bin/env python3
```

```
import sys
```

```
current_word = None
```

```
current_count = 0
```

```
for line in sys.stdin:
```

```
    word, count = line.strip().split('\t', 1)
```

```
    count = int(count)
```

```
    if current_word == word:
```

```
        current_count += count
```

```
    else:
```

```
if current_word:
    print(f"{current_word}\t{current_count}")
    current_word = word
    current_count = count
```

```
if current_word:
    print(f"{current_word}\t{current_count}")
```

Input.txt

apple banana apple cherry